

Indeksy, optymalizator Lab1

Imiona i nazwiska: Marek Małek, Mateusz Lampert

Celem ćwiczenia jest zapoznanie się z planami wykonania zapytań (execution plans), oraz z budową i możliwością wykorzystaniem indeksów.

Swoje odpowiedzi wpisuj w miejsca oznaczone jako:

Wyniki:

-- ...

Ważne/wymagane są komentarze.

Zamieść kod rozwiązania oraz zrzuty ekranu pokazujące wyniki

- dołącz kod rozwiązania w formie tekstowej/źródłowej
- najlepiej plik .md
 - ewentualnie sql

Zwróć uwagę na formatowanie kodu

Oprogramowanie - co jest potrzebne?

Do wykonania ćwiczenia potrzebne jest następujące oprogramowanie

- MS SQL Server
- SSMS - SQL Server Management Studio
 - ewentualnie inne narzędzie umożliwiające komunikację z MS SQL Server i analizę planów zapytań
- przykładowa baza danych AdventureWorks2017.

Oprogramowanie dostępne jest na przygotowanej maszynie wirtualnej

Przygotowanie

Stwórz swoją bazę danych o nazwie lab1.

```
create database lab1
go
```

```
use lab1
go
```

Część 1

Celem tej części ćwiczenia jest zapoznanie się z planami wykonania zapytań (execution plans) oraz narzędziem do automatycznego generowania indeksów.

Dokumentacja/Literatura

Przydatne materiały/dokumentacja. Proszę zapoznać się z dokumentacją:

- <https://docs.microsoft.com/en-us/sql/tools/dta/tutorial-database-engine-tuning-advisor>
- <https://docs.microsoft.com/en-us/sql/relational-databases/performance/start-and-use-the-database-engine-tuning-advisor>
- <https://www.simple-talk.com/sql/performance/index-selection-and-the-query-optimizer>
- <https://blog.quest.com/sql-server-execution-plan-what-is-it-and-how-does-it-help-with-performance-problems/>

Operatory (oraz reprezentujące je piktogramy/Ikonki) używane w graficznej prezentacji planu zapytania opisane są tutaj:

- <https://docs.microsoft.com/en-us/sql/relational-databases/showplan-logical-and-physical-operators-reference>

Wykonaj poniższy skrypt, aby przygotować dane:

```
select *
into [salesorderheader]
from [adventureworks2017].sales.[salesorderheader]
go
```

```
select *
into [salesorderdetail]
from [adventureworks2017].sales.[salesorderdetail]
go
```

Zadanie 1 - Obserwacja

Wpisz do MSSQL Managment Studio (na razie nie wykonuj tych zapytań):

```
-- zapytanie 1
select *
from salesorderheader sh
    inner join salesorderdetail sd on sh.salesorderid = sd.salesorderid
where orderdate = '2008-06-01 00:00:00.000'
go

-- zapytanie 1.1
select *
from salesorderheader sh
    inner join salesorderdetail sd on sh.salesorderid = sd.salesorderid
where orderdate = '2013-01-28 00:00:00.000'
go

-- zapytanie 2
select orderdate,
    productid,
    sum(orderqty) as orderqty,
    sum(unitpricediscount) as unitpricediscount,
    sum(linetotal)
from salesorderheader sh
    inner join salesorderdetail sd on sh.salesorderid = sd.salesorderid
group by orderdate, productid
having sum(orderqty) >= 100
go

-- zapytanie 3
select salesordernumber, purchaseordernumber, duedate, shipdate
from salesorderheader sh
    inner join salesorderdetail sd on sh.salesorderid = sd.salesorderid
where orderdate in ('2008-06-01', '2008-06-02', '2008-06-03', '2008-06-04', '2008-06-05')
go

-- zapytanie 4
select sh.salesorderid, salesordernumber, purchaseordernumber, duedate, shipdate
from salesorderheader sh
    inner join salesorderdetail sd on sh.salesorderid = sd.salesorderid
where carriertrackingnumber in ('ef67-4713-bd', '6c08-4c4c-b8')
order by sh.salesorderid
go
```

Włącz dwie opcje: **Include Actual Execution Plan** oraz **Include Live Query Statistics**:

Teraz wykonaj poszczególne zapytania (najlepiej każde analizuj oddzielnie). Co można o nich powiedzieć? Co sprawdzają? Jak można je zoptymalizować?

Wyniki:

```
-- zapytanie 1
select *
from salesorderheader sh
    inner join salesorderdetail sd on sh.salesorderid = sd.salesorderid
where orderdate = '2008-06-01 00:00:00.000'
go
```

- Live Query Statistics:

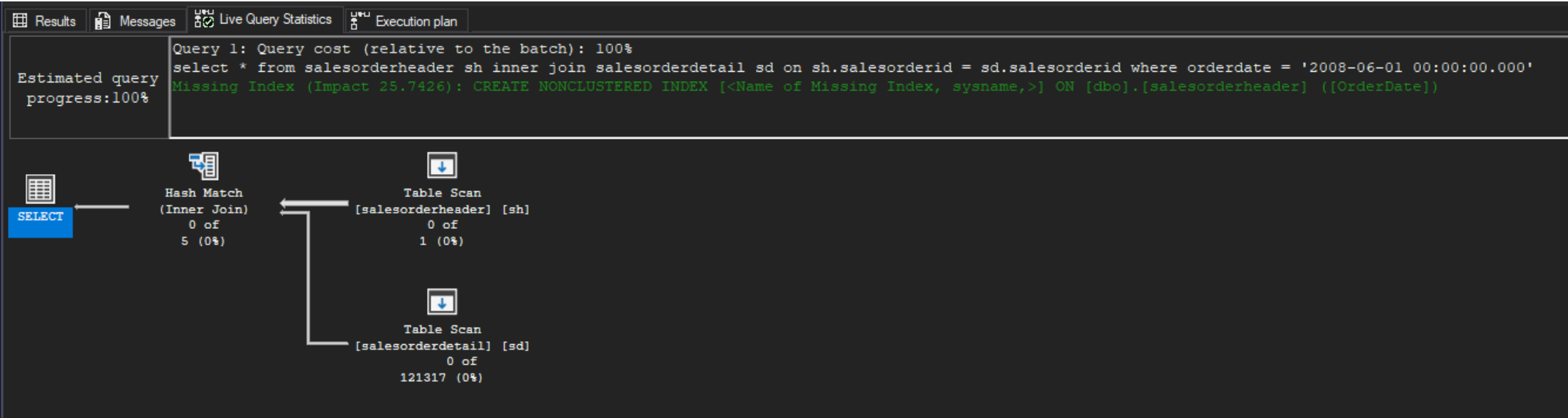


Figure 1: Live Query Statistics dla zapytania 1

- Execution Plan:

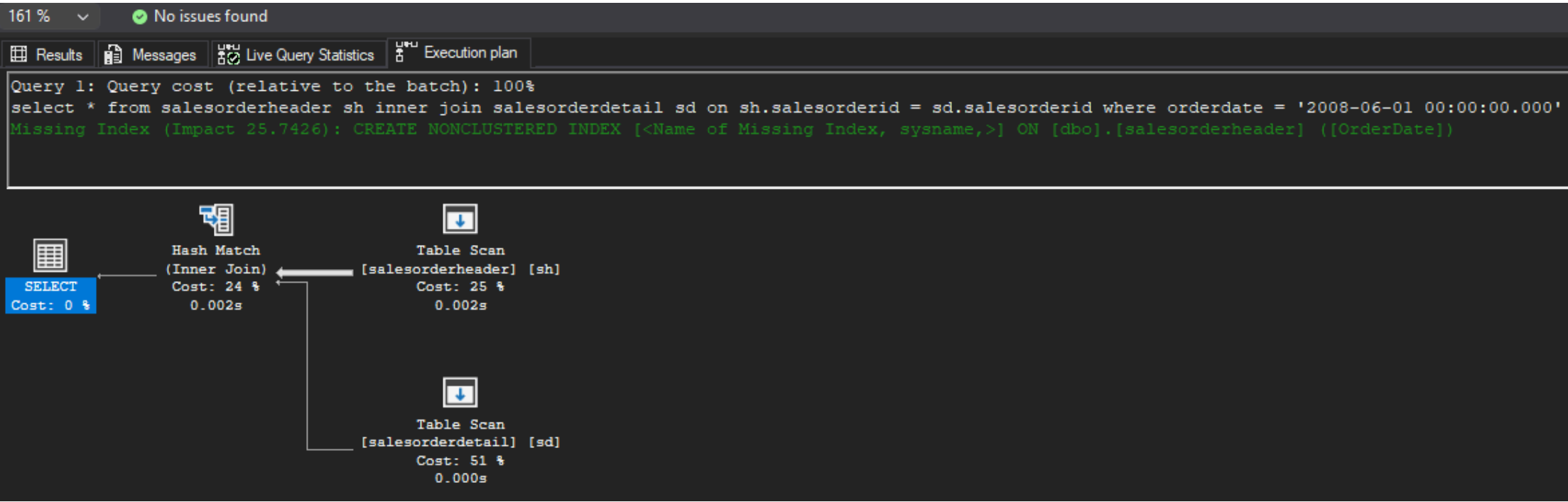


Figure 2: Execution Plan dla zapytania 1

- Wnioski:
 - najwięcej kosztu generuje skanowanie tabeli salesorderheader (Table Scan) w celu znalezienia wszystkich rekordów z datą 2008-06-01
 - to zapytanie można zoptymalizować poprzez dodanie indeksu na kolumnie orderdate w tabeli salesorderheader, co pozwoliłoby na szybsze wyszukiwanie rekordów z określoną datą (co proponuje SSMS poprzez Missing Index Suggestion z Impact ~ 25%)
 - serwer wykonuje Hash Match do połączenia tabel salesorderheader i salesorderdetail
 - podczas skanowania serwer estymuje, że zapytanie zwróci 121317 rekordów, a w rzeczywistości zwraca 0 rekordów, co mogło wpłynąć na wybór planu zapytania

```
-- zapytanie 1.1
select *
from salesorderheader sh
    inner join salesorderdetail sd on sh.salesorderid = sd.salesorderid
where orderdate = '2013-01-28 00:00:00.000'
go
```

- Live Query Statistics:

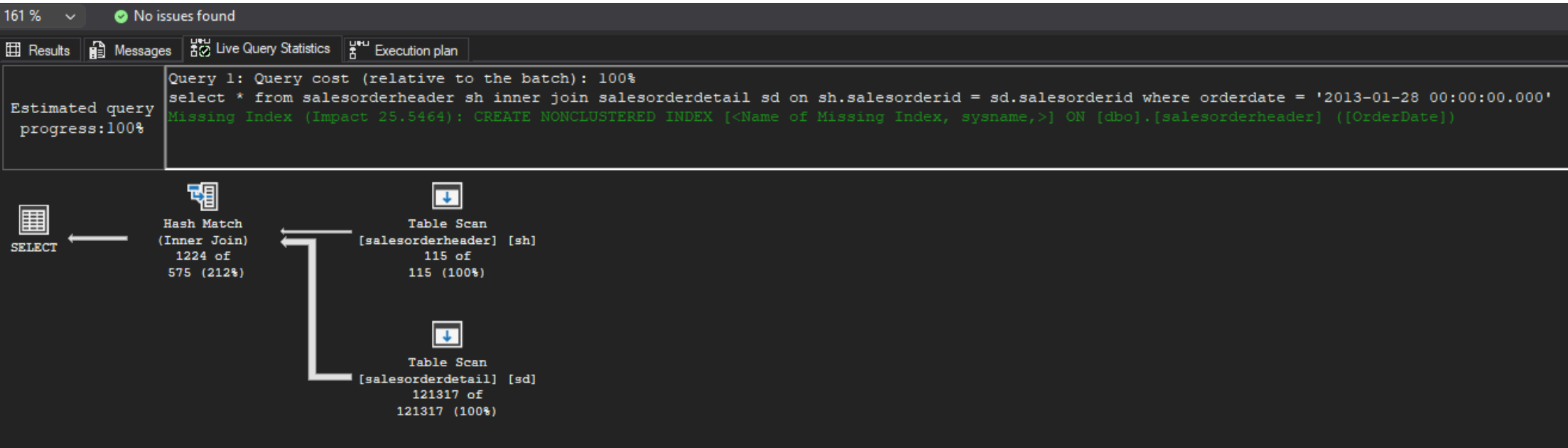


Figure 3: Live Query Statistics dla zapytania 1.1

- Execution Plan:

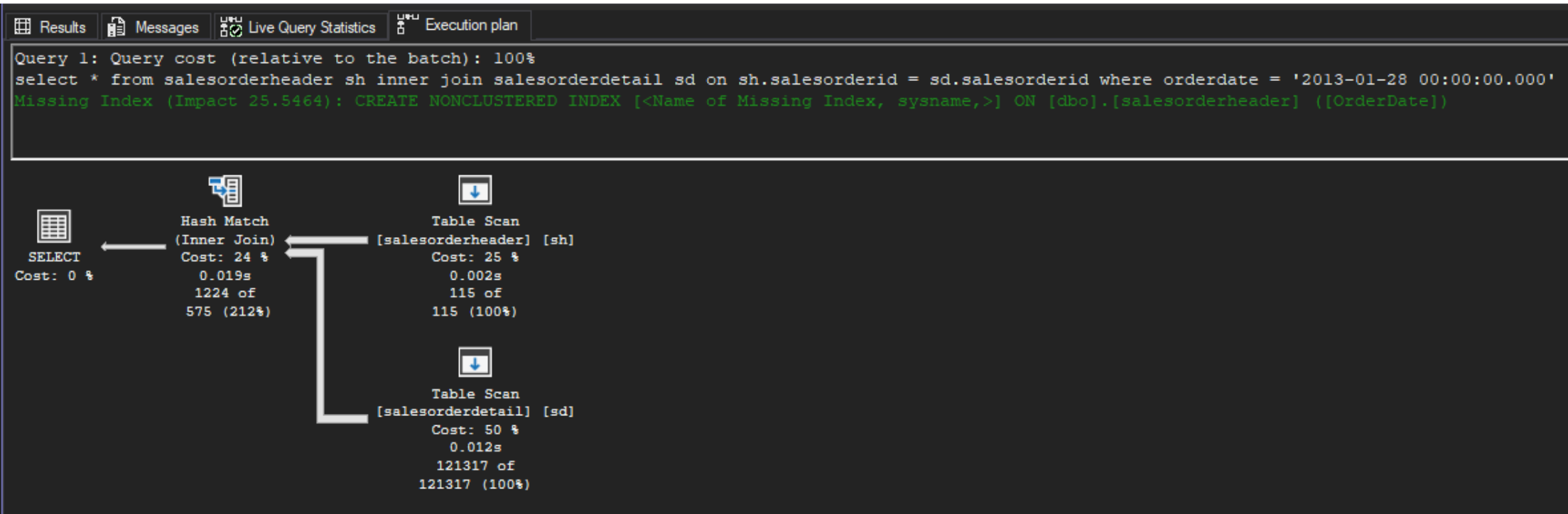


Figure 4: Execution Plan dla zapytania 1.1

Wnioski:

- znów najwięcej kosztu generuje skanowanie tabeli salesorderheader (Table Scan) w celu znalezienia wszystkich rekordów z datą 2013-01-28
- znów SSMS proponuje dodanie indeksu na kolumnie orderdate w tabeli salesorderheader z Impact ~ 25%
- serwer wykonuje Hash Match do połączenia tabel salesorderheader i salesorderdetail
- serwer estymuje, że zapytanie zwróci 575 rekordów, w rzeczywistości zwraca 1224 rekordy

```
-- zapytanie 2
select orderdate,
       productid,
       sum(orderqty)      as orderqty,
       sum(unitpricediscount) as unitpricediscount,
       sum(linetotal)
from salesorderheader sh
       inner join salesorderdetail sd on sh.salesorderid = sd.salesorderid
group by orderdate, productid
having sum(orderqty) >= 100
go
```

- Live Query Statistics:

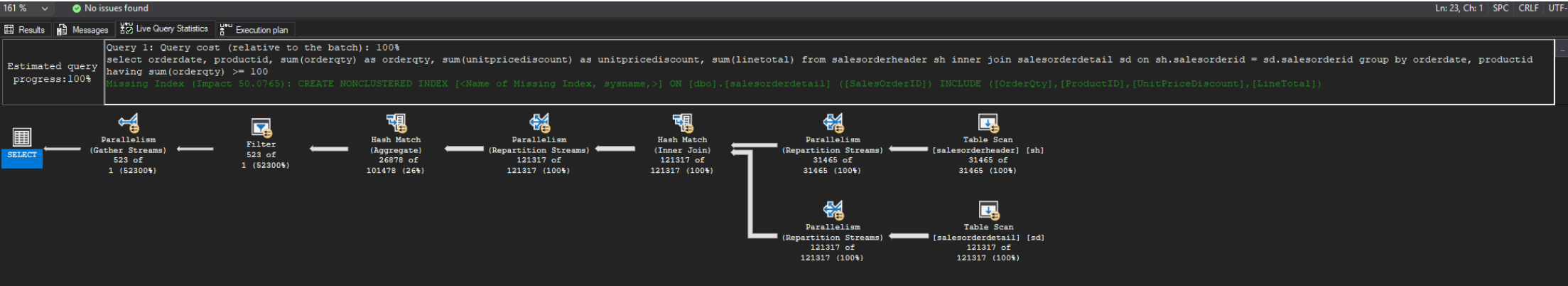


Figure 5: Live Query Statistics dla zapytania 2

- Execution Plan:

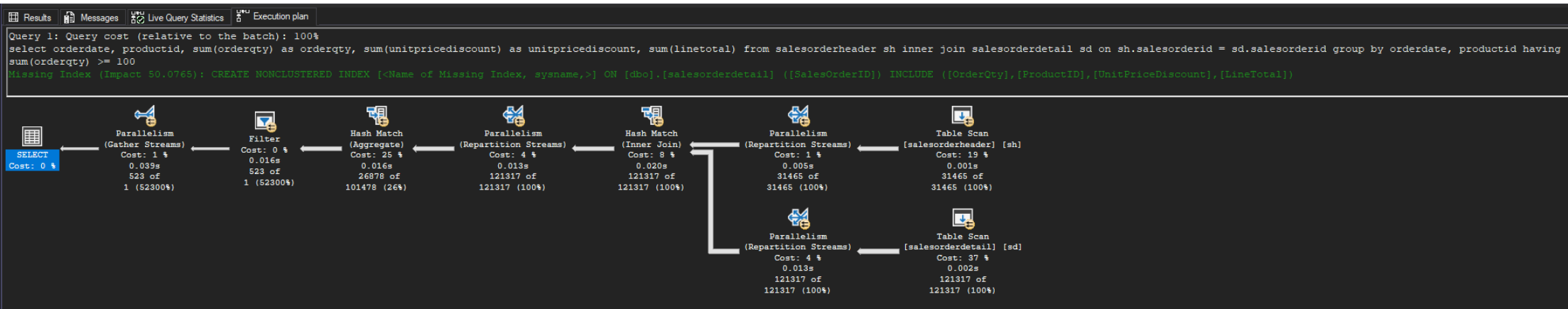


Figure 6: Execution Plan dla zapytania 2

Wnioski:

- znów najwięcej kosztu generuje skanowanie tabeli salesorderheader (Table Scan) w celu znalezienia wszystkich rekordów (kosz-y ~37%), dalej agregacja wyników (koszt ~25%)
- serwer wykonuje Hash Match do połączenia tabel salesorderheader i salesorderdetail oraz do grupowania danych
- SSMS proponuje dodanie indeksu na kolumnie orderdate w tabeli salesorderheader z Impact ~ 50%, ale w tym przypadku proponuje włączenie też innych kolumn do indeksu
- serwer wykorzystał Parallelism do wykonania zapytania, co skróciło czas jego wykonania (z XML: <QueryTimeStats CpuTime="247" ElapsedTime="36" />)
- serwer estymuje, że zapytanie zwróci 1 rekord, w rzeczywistości zwraca 523 rekordów

```
-- zapytanie 3
select salesordernumber, purchaseordernumber, duedate, shipdate
from salesorderheader sh
    inner join salesorderdetail sd on sh.salesorderid = sd.salesorderid
where orderdate in ('2008-06-01', '2008-06-02', '2008-06-03', '2008-06-04', '2008-06-05')
go
```

- Live Query Statistics:

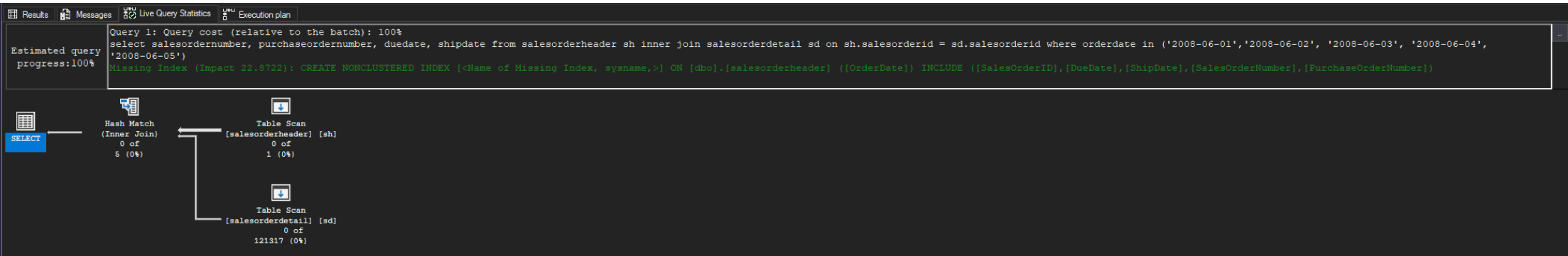


Figure 7: Live Query Statistics dla zapytania 3

- Execution Plan:

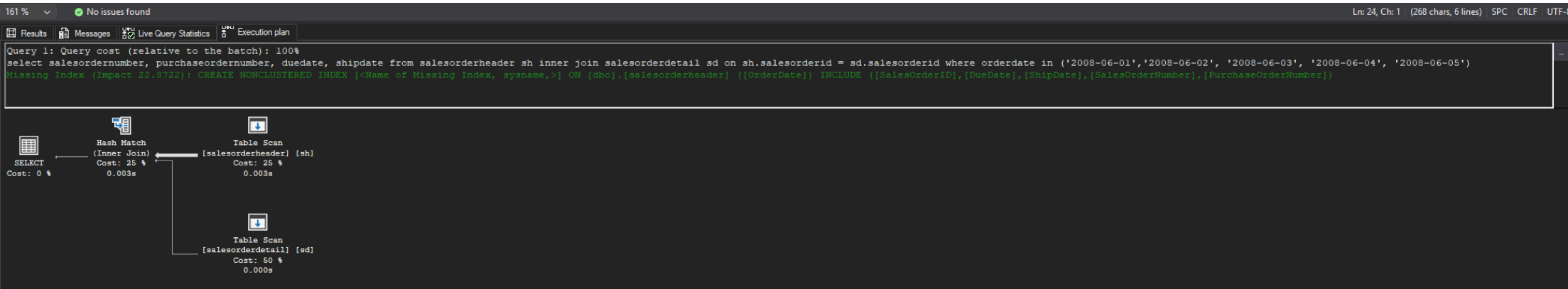


Figure 8: Execution Plan dla zapytania 3

- Wnioski:
 - znów najwięcej kosztu generuje skanowanie tabeli salesorderheader (Table Scan) w celu znalezienia wszystkich rekordów z datami z zakresu 2008-06-01 - 2008-06-05
 - serwer wykonuje Hash Match do połączenia tabel salesorderheader i salesorderdetail
 - SSMS proponuje dodanie indeksu na kolumnie orderdate w tabeli salesorderheader z Impact ~ 25% oraz włączenie innych kolumn do indeksu
 - serwer estymuje, że zapytanie zwróci 5 rekordów, w rzeczywistości zwraca 0

```
-- zapytanie 4
select sh.salesorderid, salesordernumber, purchaseordernumber, duedate, shipdate
from salesorderheader sh
    inner join salesorderdetail sd on sh.salesorderid = sd.salesorderid
where carriertrackingnumber in ('ef67-4713-bd', '6c08-4c4c-b8')
order by sh.salesorderid
go
```

- Live Query Statistics:

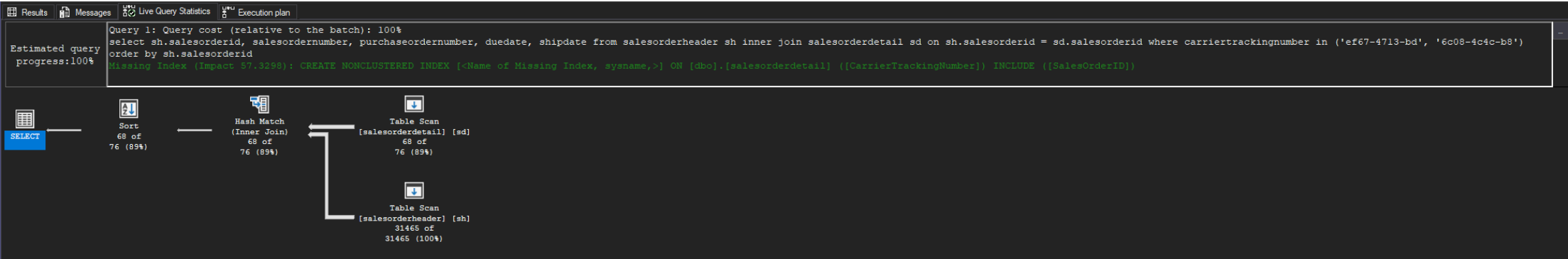


Figure 9: Live Query Statistics dla zapytania 4

- Execution Plan:

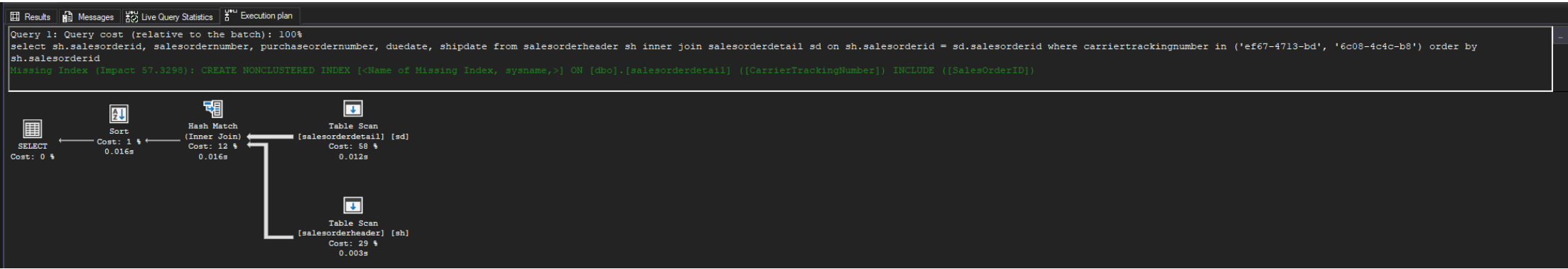


Figure 10: Execution Plan dla zapytania 4

- Wnioski:
 - znów najwięcej kosztu generuje skanowanie tabeli salesorderheader (Table Scan) w celu znalezienia wszystkich rekordów z określonymi carriertrackingnumber – w tym wypadku większy koszt generuje skanowanie tabeli predykatowej, jako, że ma ona więcej rekordow (orderdetails > orderheader)
 - serwer wykonuje Hash Match do połączenia tabel salesorderheader i salesorderdetail
 - SSMS proponuje dodanie indeksu na kolumnie carriertrackingnumber w tabeli salesorderdetail z Impact ~ 57%, włączając SalesOrderID do indeksu
 - serwer estymuje, że zapytanie zwróci 76 rekordy, a w rzeczywistości zwraca 68 rekordy

Zadanie 2 - Dobór indeksów / optymalizacja

Do wykonania tego ćwiczenia potrzebne jest narzędzie SSMS

Zapytania 1, 2, 3, 4 z poprzedniego zadania

```
-- zapytanie 1
select *
from salesorderheader sh
    inner join salesorderdetail sd on sh.salesorderid = sd.salesorderid
where orderdate = '2008-06-01 00:00:00.000'
go

-- zapytanie 2
select orderdate,
    productid,
    sum(orderqty) as orderqty,
    sum(unitpricediscount) as unitpricediscount,
    sum(linetotal)
from salesorderheader sh
    inner join salesorderdetail sd on sh.salesorderid = sd.salesorderid
group by orderdate, productid
having sum(orderqty) >= 100
go

-- zapytanie 3
select salesordernumber, purchaseordernumber, duedate, shipdate
from salesorderheader sh
    inner join salesorderdetail sd on sh.salesorderid = sd.salesorderid
where orderdate in ('2008-06-01', '2008-06-02', '2008-06-03', '2008-06-04', '2008-06-05')
go

-- zapytanie 4
select sh.salesorderid, salesordernumber, purchaseordernumber, duedate, shipdate
from salesorderheader sh
    inner join salesorderdetail sd on sh.salesorderid = sd.salesorderid
where carriertrackingnumber in ('ef67-4713-bd', '6c08-4c4c-b8')
order by sh.salesorderid
go
```

Zaznacz wszystkie zapytania, i uruchom je w **Database Engine Tuning Advisor**:

Sprawdź zakładkę **Tuning Options**, co tam można skonfigurować?

Wyniki:

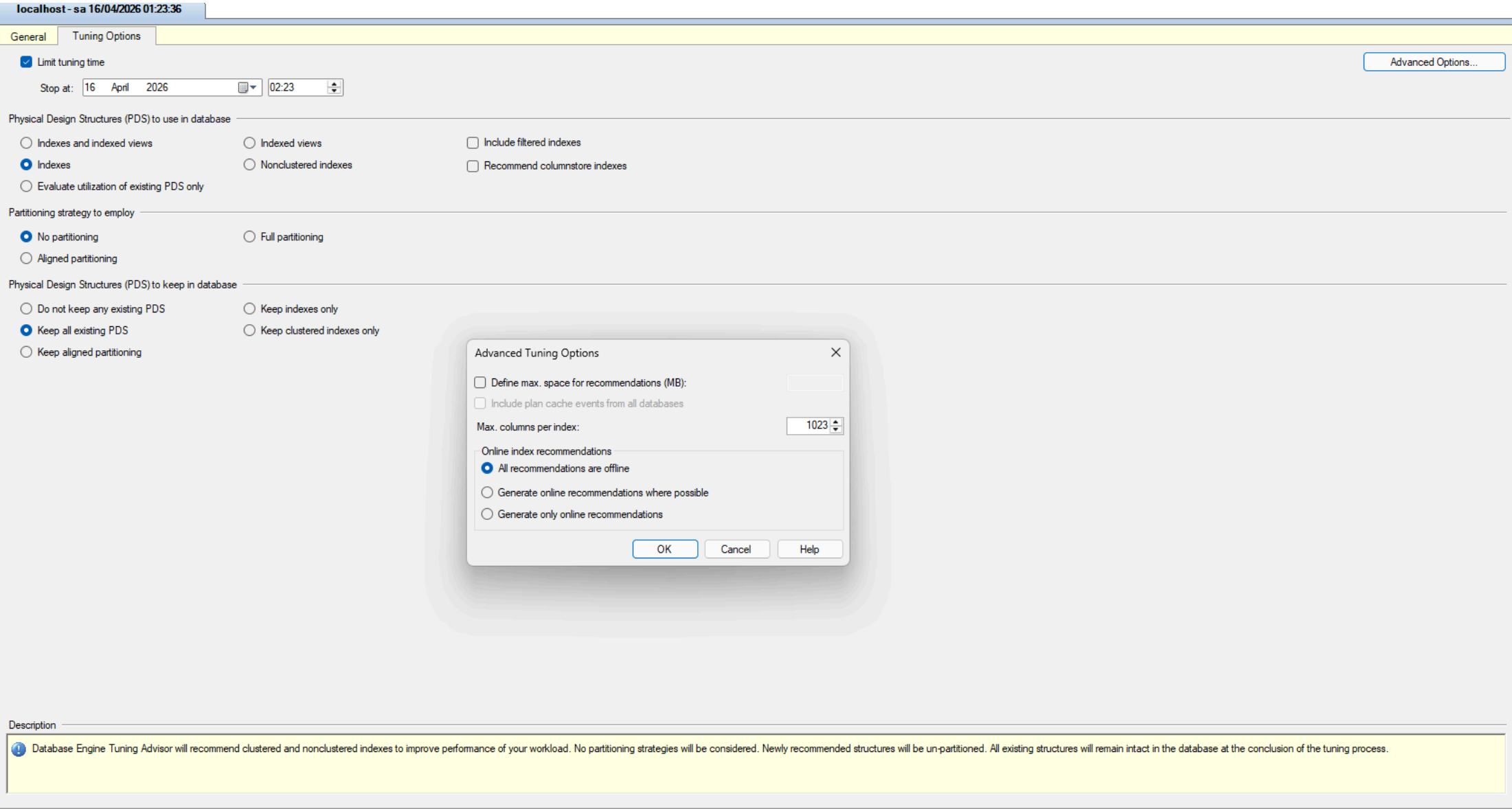


Figure 11: Tuning Options

Można tam skonfigurować:

- limit czasu analizy
- jakie fizyczne struktury mają być brane pod uwagę przy rekomendacji
- strategia partycjonowania danych
- jakie fizyczne struktury należy zachować (np. istniejące indeksy)
- maksymalne rozmiary rekomendacji
- czy rekomendacje mają być offline czy online (online - bez przerywania pracy serwera, offline - z przerwą w pracy serwera)

Użyj **Start Analysis**:

Zaobserwuj wyniki w **Recommendations**.

Przejdź do zakładki **Reports**. Sprawdź poszczególne raporty. Główną uwagę zwróć na koszty i ich poprawę:

Zapisz poszczególne rekomendacje:

Uruchom zapisany skrypt w Management Studio.

Opisz, dlaczego dane indeksy zostały zaproponowane do zapytań:

Wyniki:
Raporty:

Select report: Statement cost report			
Statement Id	Statement String	Percent Improvement	Statement Type
3	-- zapytanie 3 select salesordemumbe...	99.74	Select
1	select * from salesorderheader sh inn...	99.73	Select
4	-- zapytanie 4 select sh.salesorderid, ...	89.71	Select
2	-- zapytanie 2 select orderdate, produ...	37.85	Select

Figure 12: Statement Cost

Select report: Statement cost range report		
Cost Range	Number of statements (Current)	Number of statements (Recommended)
0% - 10%	0	3
11% - 20%	0	0
21% - 30%	0	0
31% - 40%	0	0
41% - 50%	0	0
51% - 60%	0	0
61% - 70%	0	1
71% - 80%	1	0
81% - 90%	2	0
91% - 100%	1	0

Figure 13: Statement Cost Range

Select report: Statement detail report					
Statement Id	Statement String	Statement Type	Current Statement Cost	Recommended Statement Cost	Event String
1	select * from salesorderheader sh inn...	Select	2.4611900	0.0065915	select * from salesorderheader sh inn...
2	-- zapytanie 2 select orderdate, produ...	Select	3.0575700	1.9004000	-- zapytanie 2 select orderdate, produ...
3	-- zapytanie 3 select salesordemumbe...	Select	2.5019700	0.0065915	-- zapytanie 3 select salesordemumbe...
4	-- zapytanie 4 select sh.salesorderid, ...	Select	2.1545100	0.2217650	-- zapytanie 4 select sh.salesorderid, ...

Figure 14: Statement Detail

Sprawdź jak zmieniły się Execution Plany. Opisz zmiany:

Wyniki:

```
-- zapytanie 1
select *
from salesorderheader sh
    inner join salesorderdetail sd on sh.salesorderid = sd.salesorderid
where orderdate = '2008-06-01 00:00:00.000'
go
```

- Live Query Statistics:

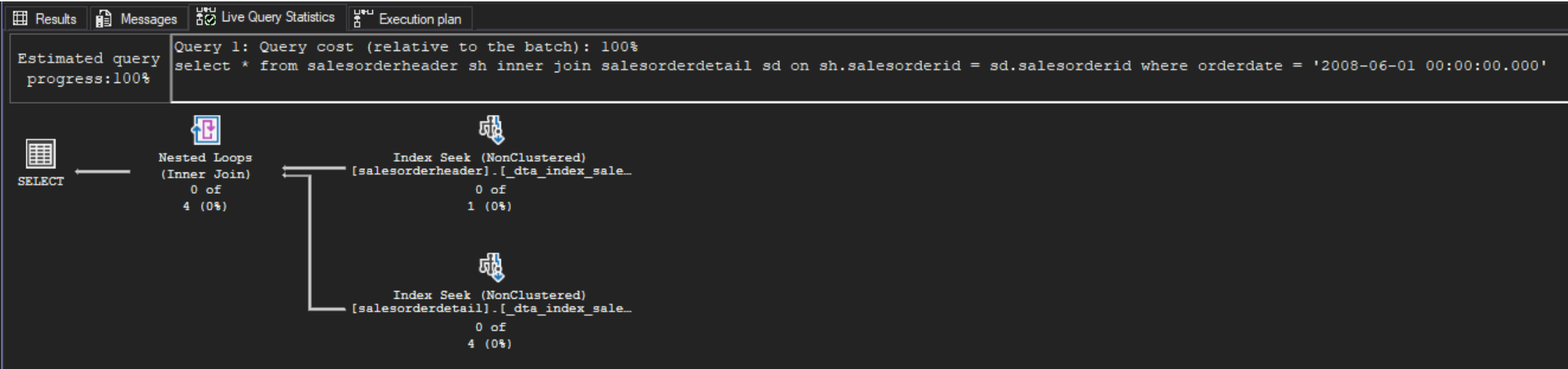


Figure 15: Live Query Statistics dla zapytania 1 po optymalizacji

- Execution Plan:

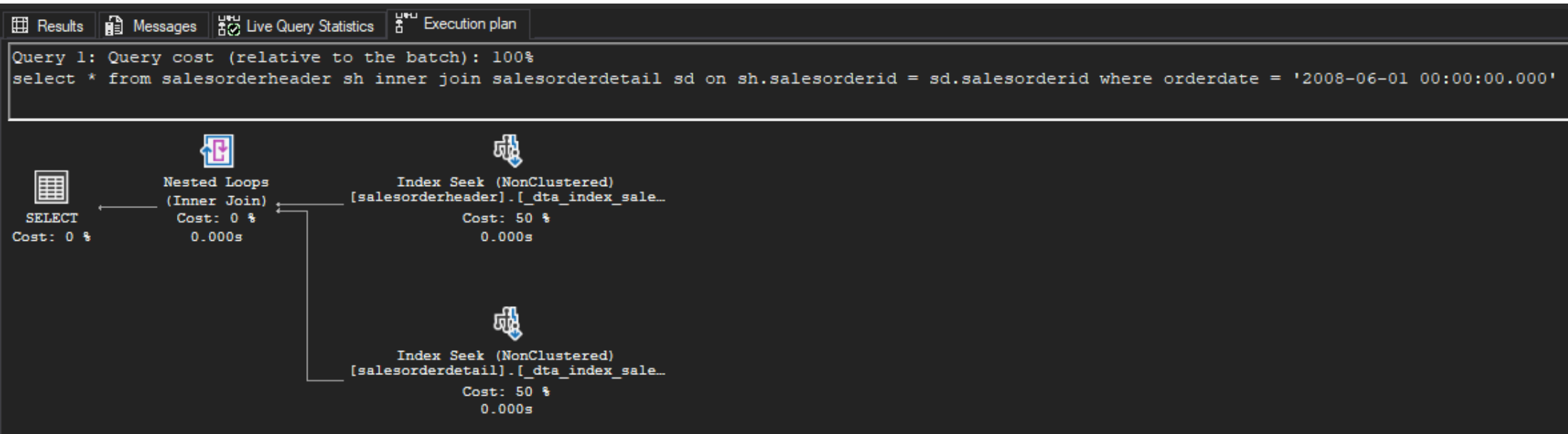


Figure 16: Execution Plan dla zapytania 1 po optymalizacji

Wnioski:

- serwer wykorzystuje Nested Loops do połączenia tabel salesorderheader i salesorderdetail
- serwer dokonuje wykorzystania indeksu orderdate w tabeli salesorderheader do wyszukania rekordów z datą 2008-06-01, co jest dużo szybsze niż skanowanie całej tabeli
- serwer o wiele lepiej estymuje liczbę zwracanych rekordów:
 - np. w przeszukiwaniu tabel 4, a zwracane jest 0 (dla porównania w poprzednim planie serwer estymował, że zwróci 121317 rekordów, a w rzeczywistości zwracał 0 rekordów)
 - w tym wypadku błąd estymacji jest mały, co sprawia, że plan jest bardziej efektywny, ponieważ serwer nie musi wykonywać dodatkowych operacji (np. Hash Match) do połączenia tabel, a może wykorzystać Nested Loops, który jest bardziej efektywny przy mniejszej liczbie zwracanych rekordów

```
-- zapytanie 2
select orderdate,
       productid,
       sum(orderqty)          as orderqty,
       sum(unitpricediscount) as unitpricediscount,
       sum(linetotal)
from salesorderheader sh
       inner join salesorderdetail sd on sh.salesorderid = sd.salesorderid
group by orderdate, productid
having sum(orderqty) >= 100
go
```

- Live Query Statistics:

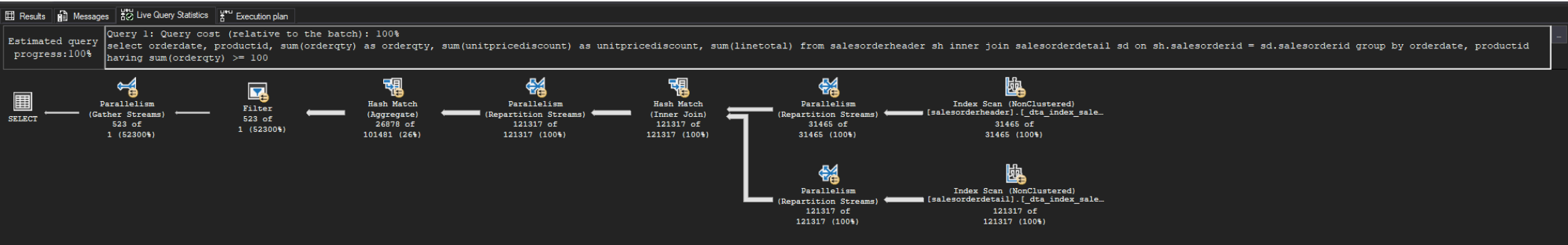
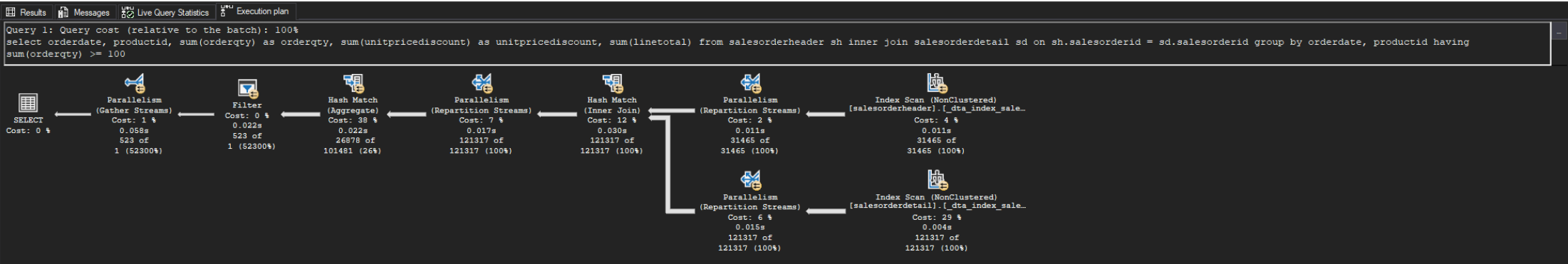


Figure 17: Live Query Statistics dla zapytania 2 po optymalizacji

- Execution Plan:



Wnioski:

- w odróżnieniu od innych zapytań, serwer wykonuje Index Scan zamiast Index Seek do wyszukania rekordów z określonymi datami, co jest spowodowane tym, że potrzebuje każdego wiersza do wyliczenia sumy
- raport wskazał, że to zapytanie najmniej skorzysta na dodaniu indeksu, co jest spowodowane tym, że nawet po optymalizacji wyszukiwania, bottleneckem pozostaje agregacja danych (koszt ~38%, najwyższy spośród wszystkich operatorów)
- zapytanie nadal wykorzystuje Parallelism do wykonania zapytania, co skróciło czas jego wykonania (z XML: <QueryTimeStats CpuTime="217" ElapsedTime="31" />)

```
-- zapytanie 3
select salesordernumber, purchaseordernumber, duedate, shipdate
from salesorderheader sh
    inner join salesorderdetail sd on sh.salesorderid = sd.salesorderid
where orderdate in ('2008-06-01', '2008-06-02', '2008-06-03', '2008-06-04', '2008-06-05')
go
```

- Live Query Statistics:

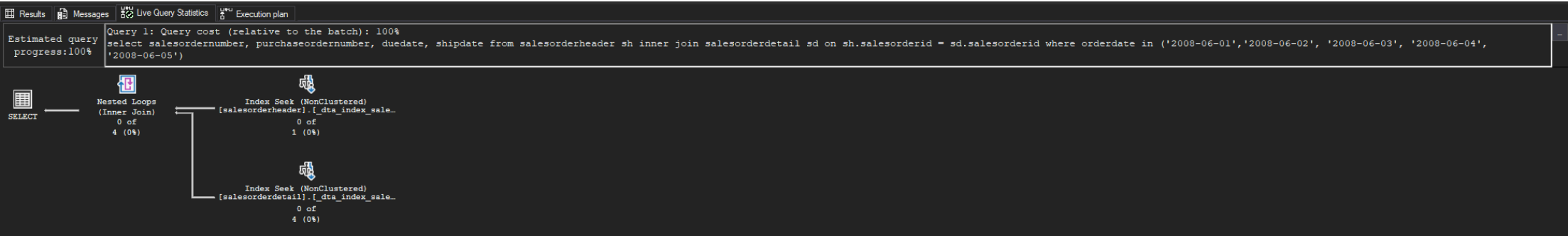


Figure 18: Live Query Statistics dla zapytania 3 po optymalizacji

- Execution Plan:

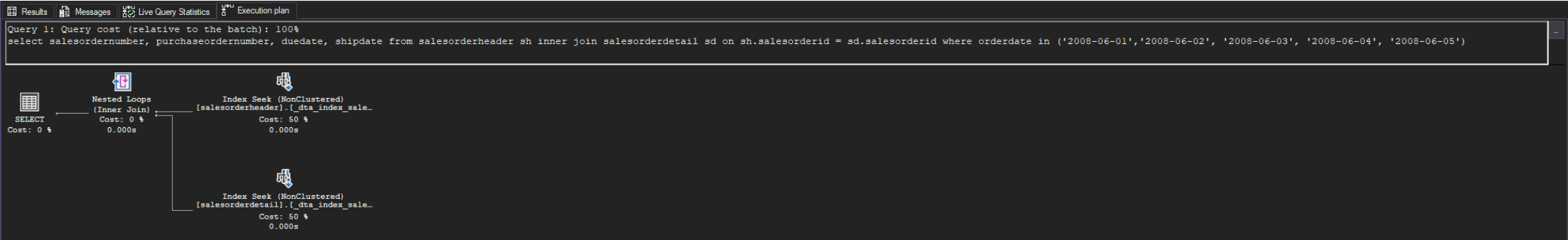


Figure 19: Execution Plan dla zapytania 3 po optymalizacji

Wnioski:

- serwer wykorzystuje Nested Loops do połączenia tabel salesorderheader i salesorderdetail
- serwer dokonuje wykorzystania indeksu orderdate w tabeli salesorderheader do wyszukania rekordów z datami z zakresu 2008-06-01 - 2008-06-05 (Index Seek), co jest dużo szybsze niż skanowanie całej tabeli
- serwer o wiele lepiej estymuje liczbę zwracanych rekordów:

```
-- zapytanie 4
select sh.salesorderid, salesordernumber, purchaseordernumber, duedate, shipdate
from salesorderheader sh
    inner join salesorderdetail sd on sh.salesorderid = sd.salesorderid
where carriertrackingnumber in ('ef67-4713-bd', '6c08-4c4c-b8')
order by sh.salesorderid
go
```

- Live Query Statistics:

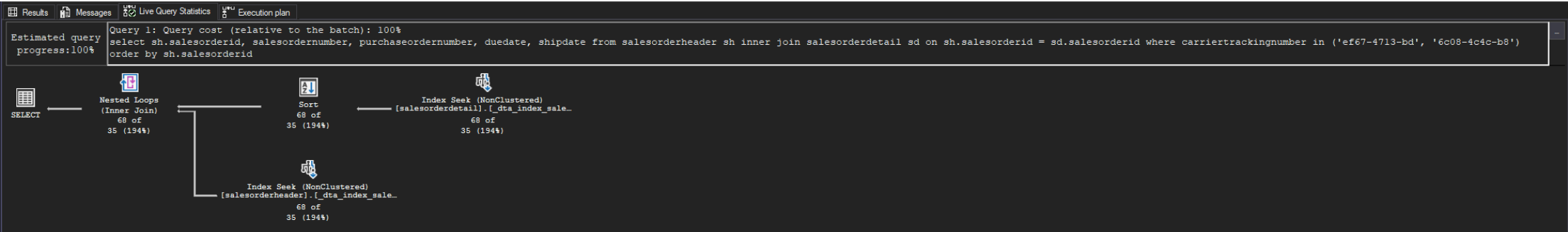


Figure 20: Live Query Statistics dla zapytania 4 po optymalizacji

- Execution Plan:

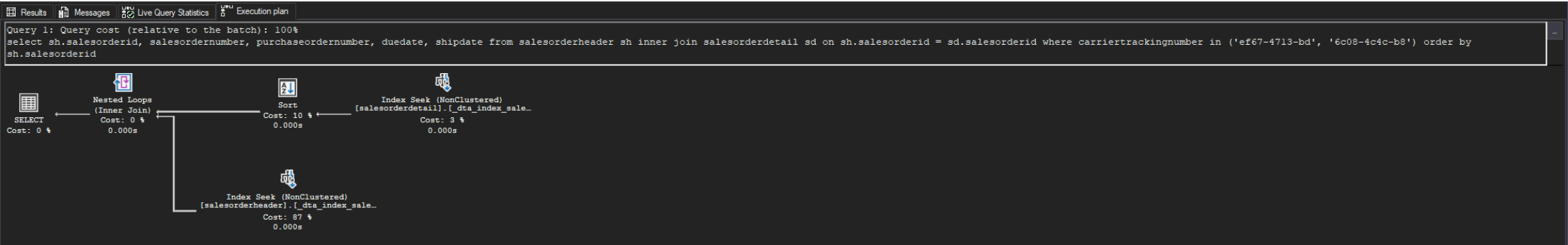


Figure 21: Execution Plan dla zapytania 4 po optymalizacji

Wnioski:

- dodanie indeksu zmieniło plan zapytania, ponieważ serwer wykonuje teraz Sort przed Nested Loops, co jest spowodowane tym, że serwerowi bardziej opłaca się posortować dane przed połączeniem tabel, niż po połączeniu tabel
- zamiast Table Scan serwer wykorzystuje Index Seek do wyszukania rekordów z określonymi carriertrackingnumber, co jest dużo szybsze niż skanowanie całej tabeli
- są błędy estymacji, ale nie są one duże, aby znacząco wpłynąć na plan zapytania

Część 2

Celem ćwiczenia jest zapoznanie się z różnymi rodzajami indeksów oraz możliwością ich wykorzystania

Dokumentacja/Literatura

Przydatne materiały/dokumentacja. Proszę zapoznać się z dokumentacją:

- <https://docs.microsoft.com/en-us/sql/relational-databases/indexes/indexes>
- <https://docs.microsoft.com/en-us/sql/relational-databases/sql-server-index-design-guide>
- <https://www.simple-talk.com/sql/performance/14-sql-server-indexing-questions-you-were-too-shy-to-ask/>
- <https://www.sqlshack.com/sql-server-query-execution-plans-examples-select-statement/>

Zadanie 3 - Indeksy klastrowane I nieklastrowane

Skopiuj tabelę Customer do swojej bazy danych:

```
select * into customer from adventureworks2017.sales.customer
```

Wykonaj analizy zapytań:

```
select * from customer where storeid = 594
```

```
select * from customer where storeid between 594 and 610
```

Zanotuj czas zapytania oraz jego koszt koszt:

Wyniki:

- zapytanie z warunkiem where storeid = 594

	CustomerID	PersonID	StoreID	TerritoryID	AccountNumber	rowguid	ModifiedDate														
1	517	NULL	594	7	AW00000517	A7726BAE-8C2D-4FE5-B25F-281083AEAF2C	2014-09-12 11:15:07.263														

	Rows	Executes	StmtText	StmtId	NodeId	Parent	PhysicalOp	LogicalOp	Argument	DefinedValues	EstimateRows	EstimateIO	EstimateCPU	AvgRowSize	TotalSubtreeCost	OutputList	Warnings	Type	Parallel	EstimateExecutions
1	1	1	SELECT * FROM [customer] WHERE [storeid]=@1	1	1	0	NULL	NULL	NULL	NULL	1	NULL	NULL	NULL	0.1391581	NULL	NULL	SELECT	0	NULL
2	1	1	--Table Scan(OBJECT:[lab1].[dbo].[customer]). ...	1	2	1	Table Scan	Table Scan	OBJECT:[lab1].[dbo].[customer]], WHERE ([lab1]...	[lab1].[dbo].[customer].[CustomerID], [lab1].[db...	1	0.1172776	0.0218805	56	0.1391581	[lab1].[dbo].[customer].[CustomerID], [lab1].[db...	NULL	PLAN_ROW	0	1

Figure 22: Plan zapytania dla warunku “where storeid=594”



Figure 23: Plan zapytania dla warunku “where storeid=594”

Time Statistics			
Client processing time	69	→	69.0000
Total execution time	96	→	96.0000
Wait time on server replies	27	→	27.0000

Figure 24: Czas wykonania zapytania dla warunku “where storeid=594”

- zapytanie z warunkiem where storeid between 594 and 610:

	CustomerID	PersonID	StoreID	TerritoryID	AccountNumber	rowguid	ModifiedDate														
1	350	NULL	610	5	AW00000350	51AF3285-9533-4172-A4B9-7E654169FB33	2014-09-12 11:15:07.263														
2	353	NULL	608	6	AW00000353	3B2E5C36-0827-4A31-B112-F347AD588822	2014-09-12 11:15:07.263														
3	356	NULL	606	8	AW00000356	C5ADD2B4-2679-46D0-B6BC-649592C7DA61	2014-09-12 11:15:07.263														
4	359	NULL	604	2	AW00000359	68E332E9-2929-4149-8086-FA8D7E4BC859	2014-09-12 11:15:07.263														
5	430	NULL	602	10	AW00000430	D18DFA6D-7D7E-4900-A0B6-78DF78E4DEEF	2014-09-12 11:15:07.263														
6	433	NULL	600	1	AW00000433	EB0B1054-FF23-4E35-BCFC-10B84B47EA47	2014-09-12 11:15:07.263														
7	436	NULL	598	4	AW00000436	506DD7E7-E6D0-49E7-ADDC-5CD6C811DD35	2014-09-12 11:15:07.263														
8	517	NULL	594	7	AW00000517	A7726BAE-8C2D-4FE5-B25F-281083AEAF2C	2014-09-12 11:15:07.263														
9	520	NULL	596	10	AW00000520	34C094D7-8C79-47A5-B192-9597826BD55D	2014-09-12 11:15:07.263														

	Rows	Executes	StmtText	StmtId	NodeId	Parent	PhysicalOp	LogicalOp	Argument	DefinedValues	EstimateRows	EstimateIO	EstimateCPU	AvgRowSize	TotalSubtreeCost	OutputList	Warnings	Type	Parallel	EstimateExecutions
1	16	1	SELECT * FROM [customer] WHERE [storeid]>=@1 AN...	1	1	0	NULL	NULL	NULL	NULL	16	NULL	NULL	NULL	0.1391581	NULL	NULL	SELECT	0	NULL
2	16	1	--Table Scan(OBJECT:[lab1].[dbo].[customer]), WHER...	1	2	1	Table Scan	Table Scan	OBJECT:[lab1].[dbo].[customer]], WHERE ([lab1]...	[lab1].[dbo].[customer].[CustomerID], [lab1].[db...	16	0.1172776	0.0218805	56	0.1391581	[lab1].[dbo].[customer].[CustomerID], [lab1].[db...	NULL	PLAN_ROW	0	1

Figure 25: Plan zapytania dla warunku “where storeid between 594 and 610”

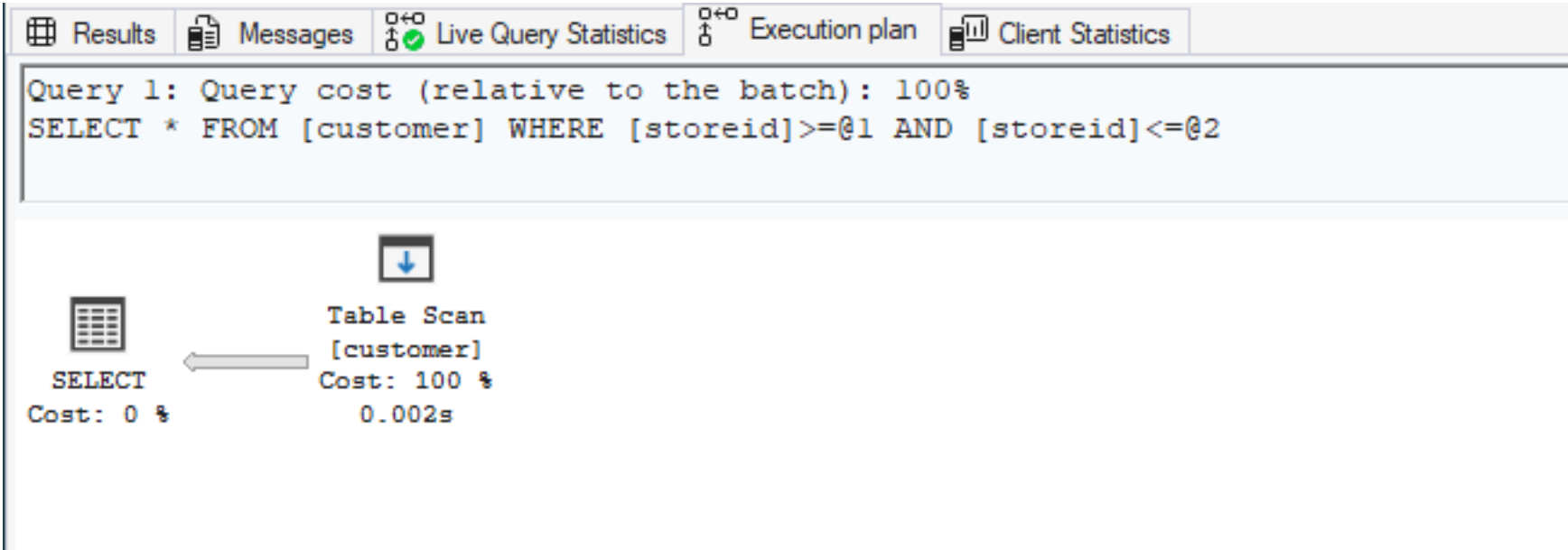


Figure 26: Plan zapytania dla warunku “where storeid between 594 and 610”

Time Statistics			
Client processing time	63	→	63.0000
Total execution time	92	→	92.0000
Wait time on server replies	29	→	29.0000

Figure 27: Czas wykonania zapytania dla warunku “where storeid between 594 and 610”

Komentarz:

Jak widać na załączonych zrzutach ekranu, w przypadku braku indeksu wykonywane jest pełny skan tabeli (wszystkie wiersze muszą zostać przeskanowane pod kątem warunku). Koszt obu zapytań jest identyczny (i tak muszą zostać przeskanowane wszystkie wiersze).

Dodaj indeks:

```
create index customer_store_cls_idx on customer(storeid)
```

Jak zmienił się plan i czas? Czy jest możliwość optymalizacji?

Wyniki:

- zapytanie z warunkiem where storeid = 594:

Results		Messages		Live Query Statistics		Execution plan		Client Statistics	
	CustomerID	PersonID	StoreID	TerritoryID	AccountNumber	rowguid	ModifiedDate		
1	517	NULL	594	7	AW00000517	A7726BAE-9C2D-4FE5-B25F-281083AEAF2C	2014-09-12 11:15:07.263		

	Rows	Executes	Stmt Text	StmtId	NodId	Parent	PhysicalOp	LogicalOp	Argument	DefinedValues	EstimateRows	EstimateIO	EstimateCPU	AvgRowSize	TotalSubtreeCost	OutputList	Warnings	Type	Parallel	EstimateExecutions
1	1	1	SELECT * FROM [customer] WHERE [storeid]=@1	1	1	0	NULL	NULL	NULL	NULL	1	NULL	NULL	NULL	0.00657038	NULL	NULL	SELECT	0	NULL
2	1	1	--Nested Loops[Inner Join, OUTER REFERENCES (...]	1	2	1	Nested Loops	Inner Join	OUTER REFERENCES ([Bnk1000])	NULL	1	0	4.18E-06	56	0.00657038	[lab 1].[dbo].[customer].[CustomerID], [lab 1].db...	NULL	PLAN_ROW	0	1
3	1	1	--Index Seek[OBJECT([lab 1].[dbo].[customer].[cu...	1	3	2	Index Seek	Index Seek	OBJECT([lab 1].[dbo].[customer].[customer_store_cl...	[Bnk1000], [lab 1].[dbo].[customer].[StoreID]	1	0.003125	0.0001581	19	0.0032831	[Bnk1000], [lab 1].[dbo].[customer].[StoreID]	NULL	PLAN_ROW	0	1
4	1	1	--RID Lookup[OBJECT([lab 1].[dbo].[customer]). ...]	1	5	2	RID Lookup	RID Lookup	OBJECT([lab 1].[dbo].[customer]). SEEK ([Bnk1000]...	[lab 1].[dbo].[customer].[CustomerID], [lab 1].db...	1	0.003125	0.0001581	52	0.0032831	[lab 1].[dbo].[customer].[CustomerID], [lab 1].db...	NULL	PLAN_ROW	0	1

Figure 28: Plan zapytania dla warunku “where storeid=594” - indeks nieklastrowany

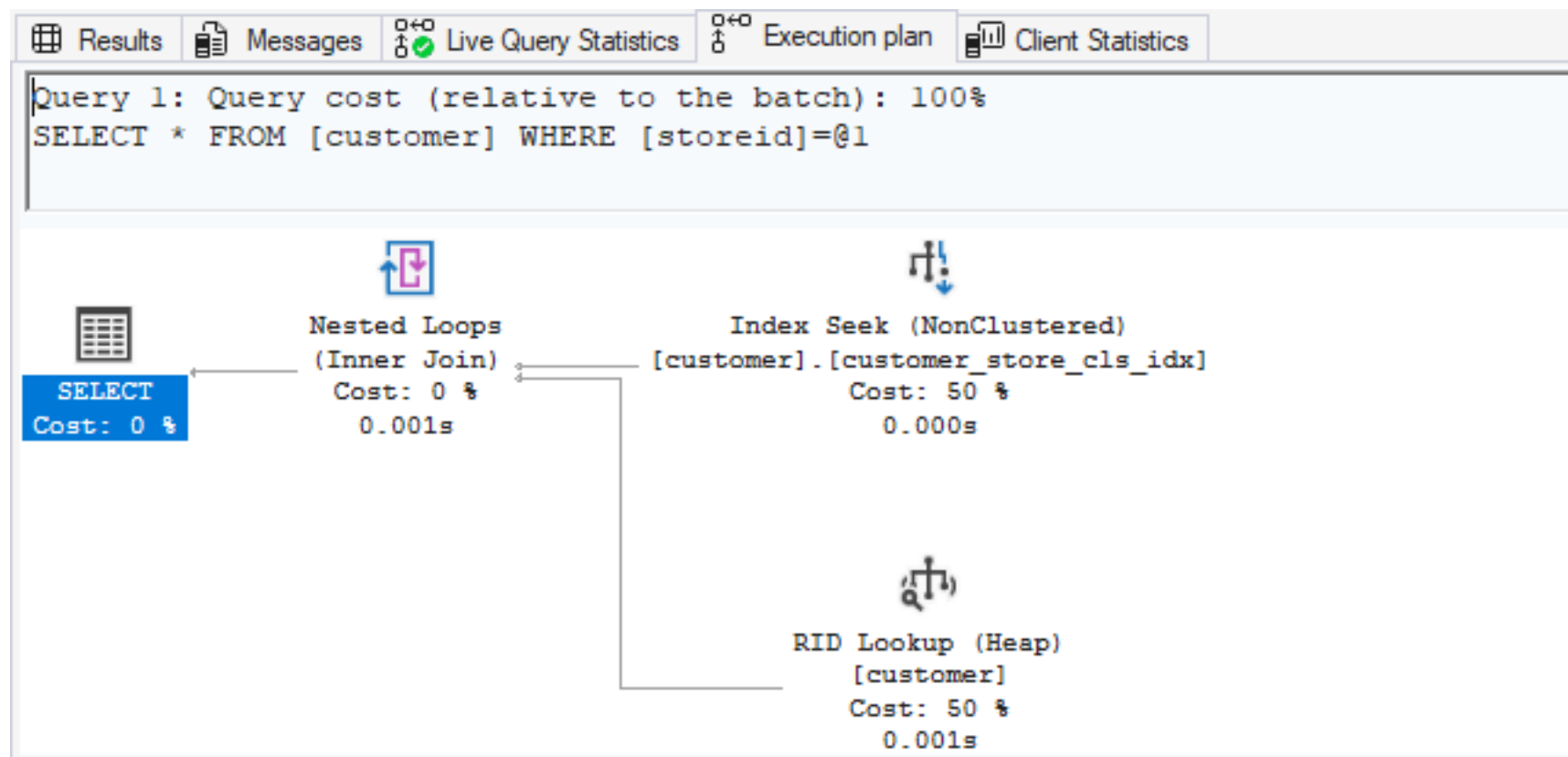


Figure 29: Plan zapytania dla warunku “where storeid=594” - indeks nieklastrowany

Time Statistics			
Client processing time	77	→	77.0000
Total execution time	115	→	115.0000
Wait time on server replies	38	→	38.0000

Figure 30: Czas wykonania zapytania dla warunku “where storeid=594” - indeks nieklastrowany

- | Results | | | | | | | | | | | | | | | Messages | Live Query Statistics | Execution plan | Client Statistics |
|---------|------------|----------|---------|-------------|---------------|--------------------------------------|-------------------------|--|--|--|--|--|--|--|----------|-----------------------|----------------|-------------------|
| | CustomerID | PersonID | StoreID | TerritoryID | AccountNumber | rowguid | ModifiedDate | | | | | | | | | | | |
| 1 | 517 | 594 | 594 | 7 | AW00000517 | A7726BAE-8C2D-4FE5-B25F-281083AEAF2C | 2014-09-12 11:15:07.263 | | | | | | | | | | | |
| 2 | 29615 | 595 | 596 | 10 | AW00029615 | C7E5D895-EABA-4CC6-A7AB-50B8569541D3 | 2014-09-12 11:15:07.263 | | | | | | | | | | | |
| 3 | 520 | 596 | 596 | 10 | AW00000520 | 34C094D7-8C79-47A5-B192-9597826BD5D | 2014-09-12 11:15:07.263 | | | | | | | | | | | |
| 4 | 29616 | 597 | 598 | 4 | AW00029616 | 21D3FF8-0A8F-434B-9290-E562B7DA6DCA | 2014-09-12 11:15:07.263 | | | | | | | | | | | |
| 5 | 436 | 598 | 598 | 4 | AW00000436 | 506DD7E7-E6D0-49E7-ADDC-5CD6C811DD35 | 2014-09-12 11:15:07.263 | | | | | | | | | | | |
| 6 | 29617 | 599 | 600 | 1 | AW00029617 | 96E154E8-8F02-4651-A117-5E248685341D | 2014-09-12 11:15:07.263 | | | | | | | | | | | |
| 7 | 433 | 600 | 600 | 1 | AW00000433 | EB0B1054-FF23-4E35-BCFC-10B84B47EA47 | 2014-09-12 11:15:07.263 | | | | | | | | | | | |
| 8 | 29618 | 601 | 602 | 10 | AW00029618 | 89BE0FA2-162C-400D-9D8B-F2E557242088 | 2014-09-12 11:15:07.263 | | | | | | | | | | | |
-
- | | Rows | Executes | Stmt Text | StmtID | NodeID | Parent | PhysicalOp | LogicalOp | Argument | DefinedValues | EstimateRows | EstimateIO | EstimateCPU | AvgRowSize | TotalSubtreeCost | OutputList | Warnings | Type | Parallel | EstimateExecutions |
|---|------|----------|---|--------|--------|--------|--------------|-------------|--|---|--------------|------------|-------------|------------|------------------|---|----------|----------|----------|--------------------|
| 1 | 16 | 1 | SELECT * FROM [customer] WHERE [storeid]>=@1 AN... | 1 | 1 | 0 | NULL | NULL | NULL | NULL | 16 | NULL | NULL | 0.0507122 | 0.0507122 | NULL | NULL | SELECT | 0 | NULL |
| 2 | 16 | 1 | I-Nested Loops(Inner Join, OUTER REFERENCES:[B... | 1 | 2 | 1 | Nested Loops | Inner Join | OUTER REFERENCES:[[Bnk1000]] | NULL | 16 | 0 | 6.688E-05 | 56 | 0.0507122 | [lab1].[dbo].[customer].[CustomerID], [lab1].[db... | NULL | PLAN_ROW | 0 | 1 |
| 3 | 16 | 1 | I-Index Seek(OBJECT:[lab1].[dbo].[customer].[custo... | 1 | 3 | 2 | Index Seek | Index Seek | OBJECT:[lab1].[dbo].[customer].[customer_store_... | [Bnk1000], [lab1].[dbo].[customer].[StoreID] | 16 | 0.003125 | 0.0001746 | 19 | 0.0032996 | [Bnk1000], [lab1].[dbo].[customer].[StoreID] | NULL | PLAN_ROW | 0 | 1 |
| 4 | 16 | 16 | I-RID Lookup(OBJECT:[lab1].[dbo].[customer].[SE... | 1 | 5 | 2 | RID Lookup | RID Look... | OBJECT:[lab1].[dbo].[customer].[SEEK:[Bnk10... | [lab1].[dbo].[customer].[CustomerID], [lab1]... | 1 | 0.003125 | 0.0001581 | 52 | 0.04734572 | [lab1].[dbo].[customer].[CustomerID], [lab1].[db... | NULL | PLAN_ROW | 0 | 16 |

Query 1: Query cost (relative to the batch): 100%

SELECT * FROM [customer] WHERE [storeid]>=@1 AND [storeid]<=@2

```
graph TD
    A[SELECT  
Cost: 0 %] --> B[Nested Loops  
(Inner Join)  
Cost: 0 %  
0.001s]
    C[Index Seek (NonClustered)  
[customer].[customer_store_cls_idx]  
Cost: 7 %  
0.000s] --> B
    D[RID Lookup (Heap)  
[customer]  
Cost: 93 %  
0.000s] --> B
```

Time Statistics			
Client processing time	75	→	75.0000
Total execution time	151	→	151.0000
Wait time on server replies	76	→	76.0000

Po dodaniu indeksu zmienił się plan zapytania - zamiast przeglądania całej tabeli, używa Nested Loops, aby dla każdego adresu znalezionej na podstawie Index Scan wykonać RID Lookup i pobrać brakujące dane z tabeli (w indeksie jest tylko storeid, resztę danych musimy pobrać z odpowiedniego miejsca w tabeli). W przypadku obu zapytań koszt zapytania jest zdecydowanie mniejszy w porównaniu do zapytania na tabeli bez indeksu (odpowiednio ~20 razy niższy w przypadku where storid=594 oraz ~2.5 razy niższe w przypadku where storeid between 594 and 610). Różnica w koszcie wynika z faktu, że w przypadku stworzonego indeksu nie mamy dostępu do pobieranych danych (w indeksie zawarte jest tylko storeid) i musimy pobrać je ze znalezionych adresów.

Dodaj indeks klastrowany:

```
create clustered index customer_store_cls_idx on customer(storeid)
```

Czy zmienił się plan/koszt/czas? Skomentuj dwa podejścia w wyszukiwaniu krotek.

Wyniki:

- zapytanie z warunkiem where storeid=594

Results										Messages										Live Query Statistics										Execution plan										Client Statistics													
CustomerID		PersonID		StoreID		TerritoryID		AccountNumber		rowguid		ModifiedDate																																									
1		517		NULL		594		7		AW00000517		A7726BAE-9C2D-4FE5-B25F-281083AEAF2C		2014-09-12 11:15:07.263																																							

Rows	Executes	StmtText	StmtId	NodeId	Parent	PhysicalOp	LogicalOp	Argument	DefinedValues	EstimateRows	EstimateIO	EstimateCPU	AvgRowSize	TotalSubtreeCost	OutputList	Warnings	Type	Parallel	EstimateExecuti
1	1	1	SELECT * FROM [customer] WHERE [storeid]=@1	1	1	0	NULL	NULL	NULL	1	NULL	NULL	NULL	0.0032831	NULL	NULL	SELECT	0	NULL
2	1	1	I-Clustered Index Seek(OBJECT:[lab 1].[dbo].[c...	1	2	1	Clustered Index Seek	Clustered Index Seek	OBJECT:[lab 1].[dbo].[customer].[idx_customer_st...	1	0.003125	0.0001581	56	0.0032831	[lab 1].[dbo].[customer].[CustomerID], [lab 1].[db...	NULL	PLAN_ROW	0	1

Figure 34: Plan zapytania dla warunku “where storeid=594” - indeks klastrowany

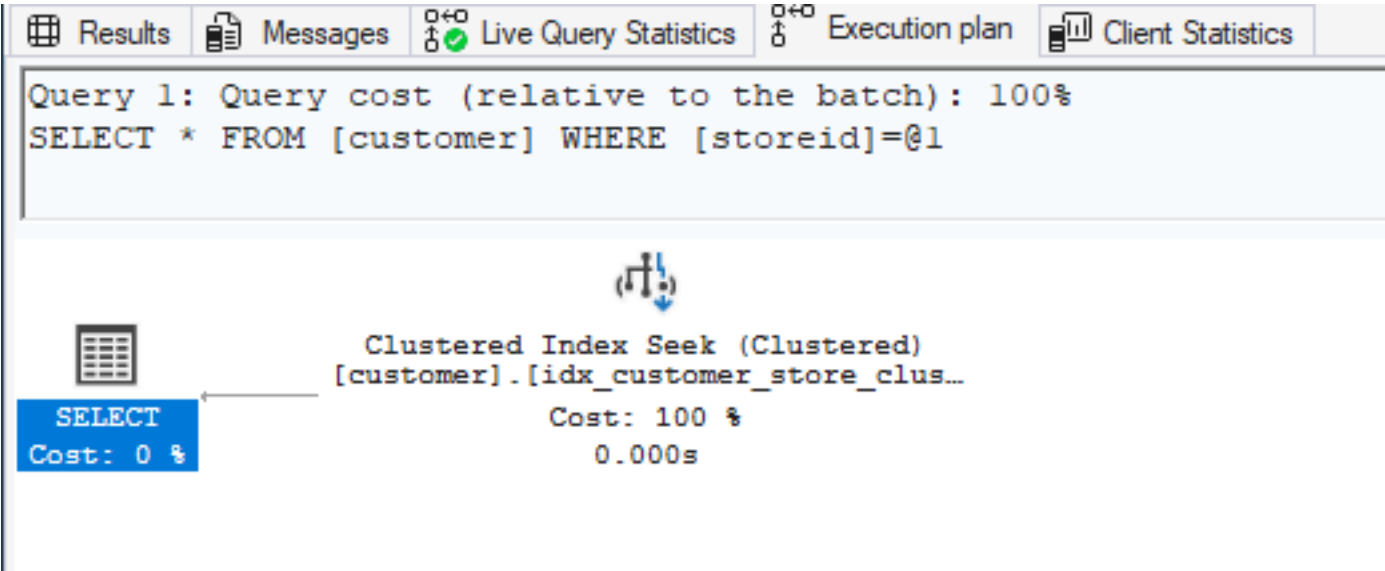


Figure 35: Plan zapytania dla warunku “where storeid=594” - indeks klastrowany

Time Statistics			
Client processing time	74	→	74.0000
Total execution time	103	→	103.0000
Wait time on server replies	29	→	29.0000

Figure 36: Czas wykonania zapytania dla warunku “where storeid=594” - indeks klastrowany

- zapytanie z warunkiem where storeid between 594 and 610:

Results		Messages		Live Query Statistics		Execution plan		Client Statistics													
CustomerID	PersonID	StoreID	TerritoryID	AccountNumber	rowguid	ModifiedDate															
1	517	NULL	594	7	AW00000517	A7726BAE-8C2D-4FE5-B25F-281083AEAF2C	2014-09-12 11:15:07.263														
2	520	NULL	596	10	AW00000520	34C094D7-9C79-47A5-B192-9597826BD55D	2014-09-12 11:15:07.263														
3	29615	595	596	10	AW00029615	C7E5D895-EABA-4CC6-A7AB-50B8569541D3	2014-09-12 11:15:07.263														
4	29616	597	598	4	AW00029616	21DF3FE8-0A8F-434B-9290-F562B7DA6DCA	2014-09-12 11:15:07.263														
Rows	Executes	StmtText	StmtId	NodeId	Parent	PhysicalOp	LogicalOp	Argument	DefinedValues	EstimateRows	EstimateIO	EstimateCPU	AvgRowSize	TotalSubtreeCost	OutputList	Warnings	Type	Parallel	Estimate		
1	16	SELECT * FROM [customer] WHERE [storeid]>=@1 AN...	1	1	0	NULL	NULL	NULL	NULL	16	NULL	NULL	NULL	0.0032996	NULL	NULL	SELECT	0	NULL		
2	16	--Clustered Index Seek(OBJECT:([lab1].[dbo].[customer]...	1	2	1	Clustered Index Seek	Clustered Index Seek	OBJECT:([lab1].[dbo].[customer].[idx_customer_st...	[lab1].[dbo].[customer].[CustomerID], [lab1].[db...	16	0.003125	0.0001746	56	0.0032996	[lab1].[dbo].[customer].[CustomerID], [lab1].[db...	NULL	PLAN_ROW	0	1		

Figure 37: Plan zapytania dla warunku “where storeid between 594 and 610” - indeks klastrowany

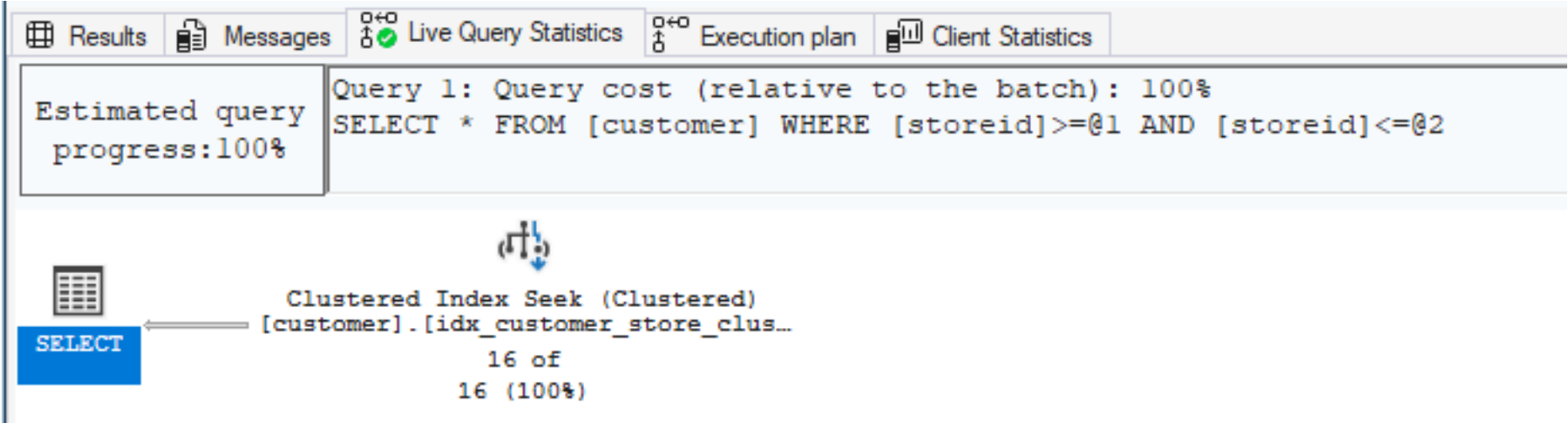


Figure 38: Plan zapytania dla warunku “where storeid between 594 and 610” - indeks klastrowany

Time Statistics			
Client processing time	85	→	85.0000
Total execution time	107	→	107.0000
Wait time on server replies	22	→	22.0000

Figure 39: Czas wykonania zapytania dla warunku “where storeid between 594 and 610” - indeks klastrowany

Komentarz:

Plan zapytania ponownie się zmienił, ponieważ stworzyliśmy indeks klastrowany ze względu na storeid mamy bezpośredni dostęp do wszystkich pól (dane zostały fizycznie przeorganizowane). Koszt w przypadku obu zapytań jest praktycznie identyczny, a także jest około:

- ~2-krotnie niższy niż zapytanie z indeksem nieklastrowanym oraz ~40-krotnie niższy niż zapytanie bez indeksu dla warunku where storeid=594
- ~10-krotnie niższy niż zapytanie z indeksem nieklastrowanym oraz ~40-krotnie niższy niż zapytanie bez indeksu dla warunku where storeid between 594 and 610

Różnice w koszcie wynikają z faktu, że w przypadku indeksu klastrowanego, dane są fizycznie reorganizowane na dysku, w rezultacie czego mamy bezpośredni dostęp do wszystkich atrybutów i nie musimy pobierać brakujących danych spod konkretnego adresu.

W przypadku warunku where storeid between 594 and 610 przewaga indeksu klastrowanego jest jeszcze większa, ponieważ dane te leżą fizycznie obok siebie na dysku.

Zadanie 4 - dodatkowe kolumny w indeksie

Celem zadania jest porównanie indeksów zawierających dodatkowe kolumny.

Skopiuj tabelę Address do swojej bazy danych:

```
select * into address from adventureworks2017.person.address
```

W tej części będziemy analizować następujące zapytanie:

```
select addressline1, addressline2, city, stateprovinceid, postalcode
from address
where postalcode between '98000' and '99999'

create index address_postalcode_1
on address (postalcode)
include (addressline1, addressline2, city, stateprovinceid);
go

create index address_postalcode_2
on address (postalcode, addressline1, addressline2, city, stateprovinceid);
go
```

Czy jest widoczna różnica w planach/kosztach zapytań?

- w sytuacji gdy nie ma indeksów
- przy wykorzystaniu indeksu:
 - address_postalcode_1
 - address_postalcode_2

Jeśli tak to jaka?

Aby wymusić użycie indeksu użyj WITH(INDEX(Address_PostalCode_1)) po FROM

```
select addressline1, addressline2, city, stateprovinceid, postalcode
from address WITH (INDEX (Address_PostalCode_1))
where postalcode between '98000' and '99999'
```

```
select addressline1, addressline2, city, stateprovinceid, postalcode
from address WITH (INDEX (Address_PostalCode_2))
where postalcode between '98000' and '99999'
```

Wyniki:

- bez indeksu:

Results		Messages		Live Query Statistics		Execution plan		Client Statistics	
addressline1		addressline2		city		stateprovinceid		postalcode	
1	1970 Napa Ct.	NULL		Bothell		79		98011	
2	9833 Mt. Dias Blv.	NULL		Bothell		79		98011	
3	7484 Roundtree Drive	NULL		Bothell		79		98011	
4	9539 Glenside Dr	NULL		Bothell		79		98011	
5	1226 Shoe St.	NULL		Bothell		79		98011	
6	1399 Firestone Drive	NULL		Bothell		79		98011	

Rows	Executes	StmtText	StmtId	NodeId	Parent	PhysicalOp	LogicalOp	Argument	DefinedValues	EstimateRows	EstimateIO	EstimateCPU	AvgRowSize	TotalSubtreeCost	OutputList	Warnings	Type	Parallel	EstimateExecutions
1	2638	1	SELECT [addressline1].[addressline2].[city].[stat...	1	1	0	NULL	NULL	NULL	2693.25	NULL	NULL	NULL	0.2781907	NULL	NULL	SELECT	0	NULL
2	2638	1	-Table Scan(OBJECT:([ab1].[dbo].[address]), ...	1	2	1	Table Scan	Table Scan	OBJECT:([ab1].[dbo].[address]), WHERE ([ab1].[...	2693.25	0.2565368	0.0216539	180	0.2781907	[ab1].[dbo].[address].[AddressLine1], [ab1].[d...	NULL	PLAN_ROW	0	1

Figure 40: Plan zapytania dla warunku - brak indeksu

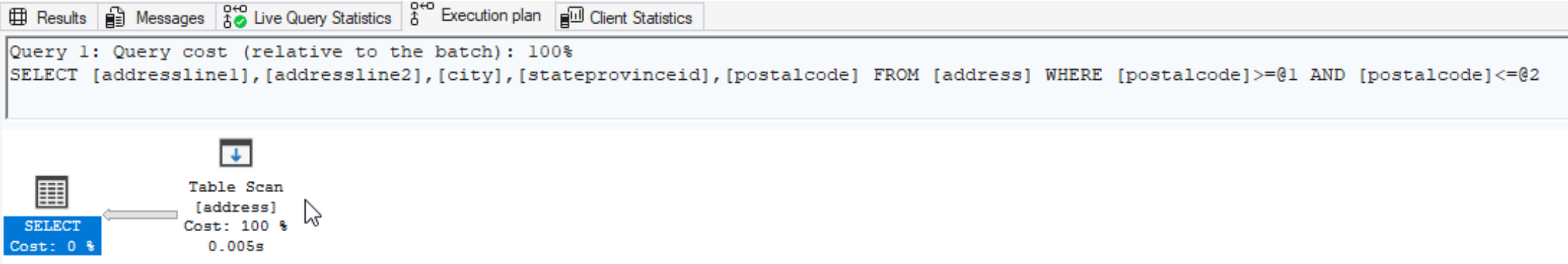


Figure 41: Plan zapytania dla warunku - brak indeksu

Time Statistics			
Client processing time	77	→	77.0000
Total execution time	138	→	138.0000
Wait time on server replies	61	→	61.0000

Figure 42: Czas wykonania zapytania dla warunku - brak indeksu

- z indeksem address_postalcode_1:

Results		Messages	Live Query Statistics	Execution plan	Client Statistics
addressline1		addressline2	city	stateprovinceid	postalcode
1	225 South 314th Street	NULL	Federal Way	79	98003
2	8684 Military East	NULL	Bellevue	79	98004
3	7270 Pepper Way	NULL	Bellevue	79	98004

Rows	Executes	Stmt Text	StmtId	NodeId	Parent	PhysicalOp	LogicalOp	Argument	DefinedValues	EstimateRows	EstimateIO	EstimateCPU	AvgRowSize	TotalSubtreeCost	OutputList	Warnings	Type	Parallel	EstimateExecutions
1	2638	1	select addressline1, addressline2, city, stateprov...	1	1	0	NULL	NULL	NULL	2693.25	NULL	NULL	NULL	0.0284668	NULL	NULL	SELECT	0	NULL
2	2638	1	--Index Seek(OBJECT:([lab1].[dbo].[address].[...	1	2	1	Index Seek	Index Seek	OBJECT:([lab1].[dbo].[address].[address_postalco...	2693.25	0.02534722	0.003119575	180	0.0284668	[lab1].[dbo].[address].[AddressLine1], [lab1].[d...	NULL	PLAN_ROW	0	1

Figure 43: Plan zapytania dla warunku - indeks nr 1

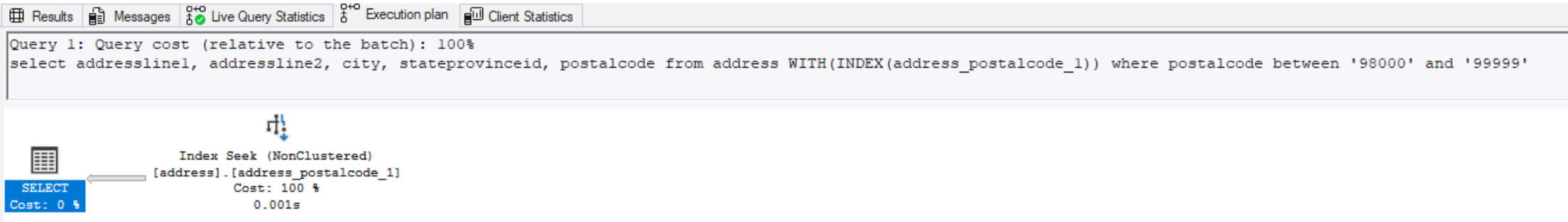


Figure 44: Plan zapytania dla warunku - indeks nr 1

Time Statistics			
Client processing time	63	→	63.0000
Total execution time	90	→	90.0000
Wait time on server replies	27	→	27.0000

Figure 45: Czas wykonania zapytania dla warunku - indeks nr 1

- z indeksem address_postalcode_2:

	addressline1	addressline2	city	stateprovinceid	postalcode																		
1	225 South 314th Street	NULL	Federal Way	79	98003																		
2	108 Lakeside Court	NULL	Bellevue	79	98004																		
3	1343 Prospect St	NULL	Bellevue	79	98004																		
Rows	Executes	StmtText				StmtId	NodeId	Parent	PhysicalOp	LogicalOp	Argument	DefinedValues	EstimateRows	EstimateIO	EstimateCPU	AvgRowSize	TotalSubtreeCost	OutputList	Warnings	Type	Parallel	EstimateExecutions	
1	2638	1	select addressline1, addressline2, city, stateprov...				1	1	0	NULL	NULL	NULL	NULL	2693.25	NULL	NULL	NULL	0.0284668	NULL	NULL	SELECT	0	NULL
2	2638	1	--Index Seek(OBJECT:([lab1].[dbo].[address][...				1	2	1	Index Seek	Index Seek	OBJECT:([lab1].[dbo].[address].[address_postalco...	[lab1].[dbo].[address].[AddressLine1], [lab1].[d...	2693.25	0.02534722	0.003119575	180	0.0284668	[lab1].[dbo].[address].[AddressLine1], [lab1].[d...	NULL	PLAN_ROW	0	1

Figure 46: Plan zapytania dla warunku - indeks nr 2

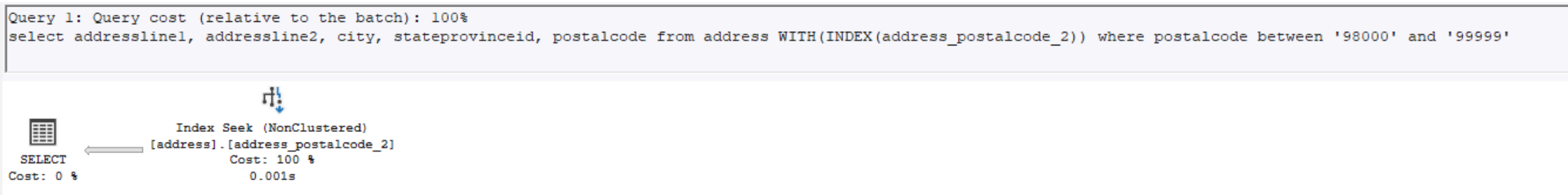


Figure 47: Plan zapytania dla warunku - indeks nr 2

Time Statistics			
Client processing time	58	→	58.0000
Total execution time	83	→	83.0000
Wait time on server replies	25	→	25.0000

Figure 48: Czas wykonania zapytania dla warunku - indeks nr 2

Komentarz:

Pomiędzy zapytaniem bez indeksu a zapytaniami korzystającymi z indeksów występuje znacząca różnica w planach zapytania (zapytanie bez indeksu wykonuje pełne przeszukiwanie tabeli, zapytania z indeksami skanuje tylko indeks, a ponieważ oba indeksy pokrywają zapytanie to nie ma konieczności dodatkowego pobierania brakujących danych). Zapytania korzystające z indeksu address_postalcode_1 oraz address_postalcode_2 mają de facto identyczne plany zapytań (koszt również jest identyczny).

Sprawdź rozmiar Indeksów:

```
select i.name as indexname, sum(s.used_page_count) * 8 as indexsizekb
from sys.dm_db_partition_stats as s
      inner join sys.indexes as i on s.object_id = i.object_id and s.index_id = i.index_id
where i.name = 'address_postalcode_1'
      or i.name = 'address_postalcode_2'
group by i.name
go
```

Który jest większy? Jak można skomentować te dwa podejścia do indeksowania? Które kolumny na to wpływają?

Wyniki:

	indexname	indexsizekb
1	address_postalcode_1	1784
2	address_postalcode_2	1808

Figure 49: Rozmiary indeksów “address_postalcode_1” oraz “address_postalcode_2”

Komentarz:

Indeks address_postalcode_2 jest nieznacznie większy niż indeks address_postalcode_1 - wynika to z faktu, że w indeksie nr 1 kolumny addressline1, addressline2, city oraz stateprovinceid znajdują się wyłącznie na poziomie liści (indeks uwzględnia tylko postalcode, ale mamy bezpośredni dostęp do pozostałych kolumn), natomiast w indeksie nr 2 kolumny addressline1, addressline2, city oraz stateprovinceid są częścią klucza, a więc muszą one być uwzględnione na wszystkich poziomach drzewa indeksu.

W przypadku naszego zapytania indeks address_postalcode_2 nie daje nam znaczącej przewagi, natomiast byłby on dużo bardziej wydajny niż indeks nr 1 np. przypadku filtrowania po większej liczbie kolumn (np. where postalcode="..." and city="...").

Zadanie 5 - kolejność atrybutów

Skopiuj tabelę Person do swojej bazy danych:

```
select businessentityid
      ,persontype
      ,namestyle
      ,title
      ,firstname
      ,middlename
      ,lastname
      ,suffix
      ,emailpromotion
      ,rowguid
      ,modifieddate
into person
from adventureworks2017.person.person
```

Wykonaj analizę planu dla trzech zapytań:

```
select * from [person] where lastname = 'Agbonile'

select * from [person] where lastname = 'Agbonile' and firstname = 'Osarumwense'

select * from [person] where firstname = 'Osarumwense'
```

Co można o nich powiedzieć?

Wyniki:

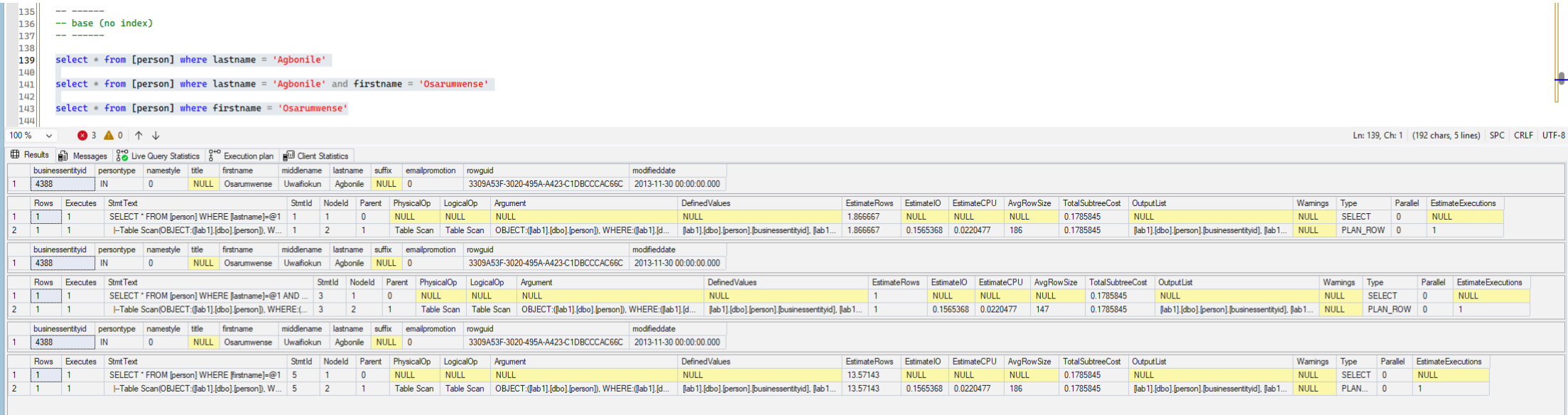


Figure 50: Plany zapytań dla zapytań o Osarumwese Agbonile - brak indeksu

Komentarz:

W przypadku wszystkich zapytań korzystamy z pełnego przeszukiwania (full scan), ponieważ nie mamy indeksu. Niezależnie od kolejności atrybutów, koszt zapytania jest identyczny, ponieważ i tak musimy przejrzeć wszystkie wiersze.

Przygotuj indeks obejmujący te zapytania:

```
create index person_first_last_name_idx
on person(lastname, firstname)
```

Sprawdź plan zapytania. Co się zmieniło?

Wyniki:

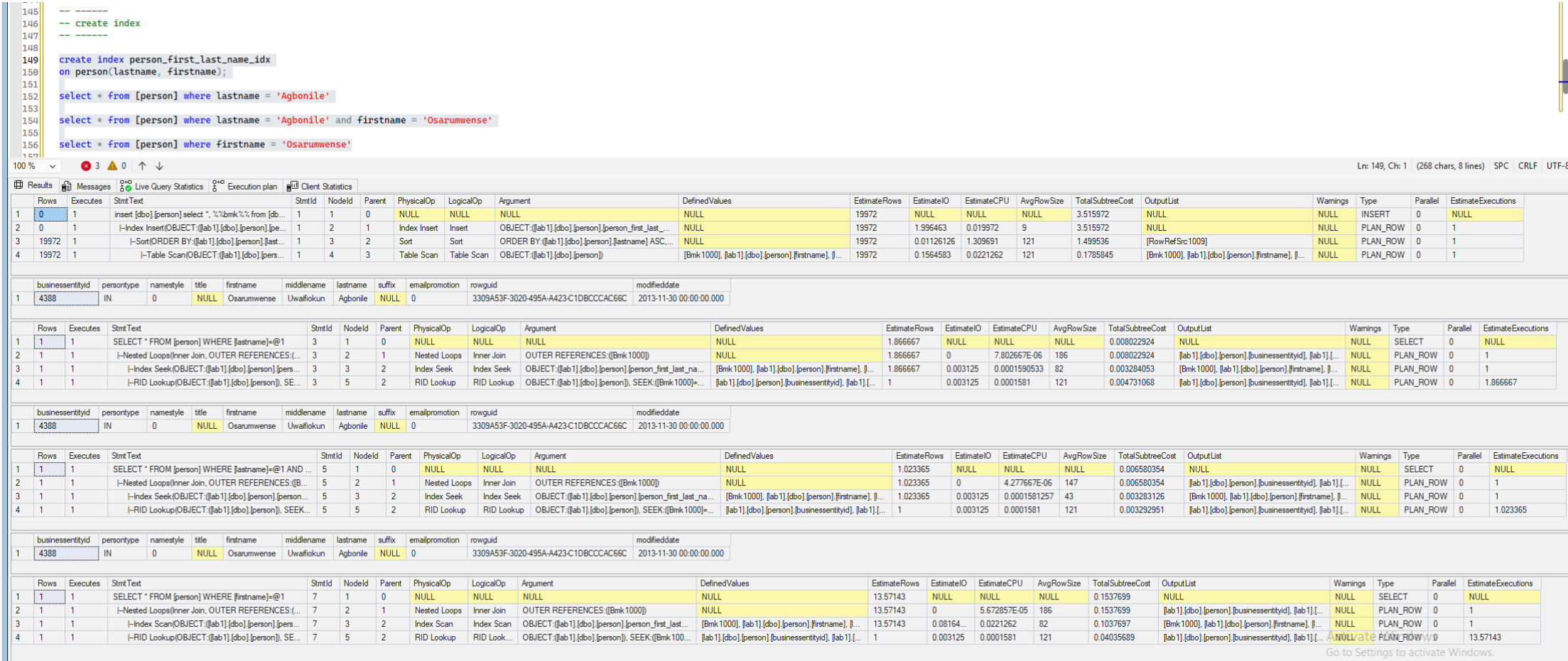


Figure 51: Plany zapytań o Osarumwese Agbonile - indeks na (lastname, firstame)

Komentarz:

- w przypadku zapytań korzystających z filtra where lastname="Agbonile" zapytania korzystają z bardzo wydajnej operacji Index Seek, która pozwala na dostęp
- w przypadku zapytania korzystającego jedynie z filtra where firstname=... , zapytanie nie może korzystać z Index Seek, a zamiast tego wykorzystywana jest operacja Index Scan - wynika to z tego, że “pominęliśmy” jeden poziom w indeksie (klauzula nie pozwala na precyzyjne wyznaczenie “ścieżki” do danych i w rezultacie musimy przeszukać cały indeks)
- zapytanie korzystające z obu atrybutów w klauzuli where charakteryzuje się najniższym kosztem (~0.006 w porównaniu do ~0.008 dla zapytanie korzystające wyłącznie z atrybutu lastname). Zapytanie korzystające jedynie z atrybutu firstname charakteryzuje się najwyższym kosztem i ma jedynie nieznacznie mniejszy koszt niż zapytanie bez indeksu.

Przeprowadź ponownie analizę zapytań tym razem dla parametrów: FirstName = ‘Angela’ LastName = ‘Price’. (Trzy zapytania, różna kombinacja parametrów).

Czym różni się ten plan od zapytania o 'Osarumwense Agbonile' . Dlaczego tak jest?

Wyniki:

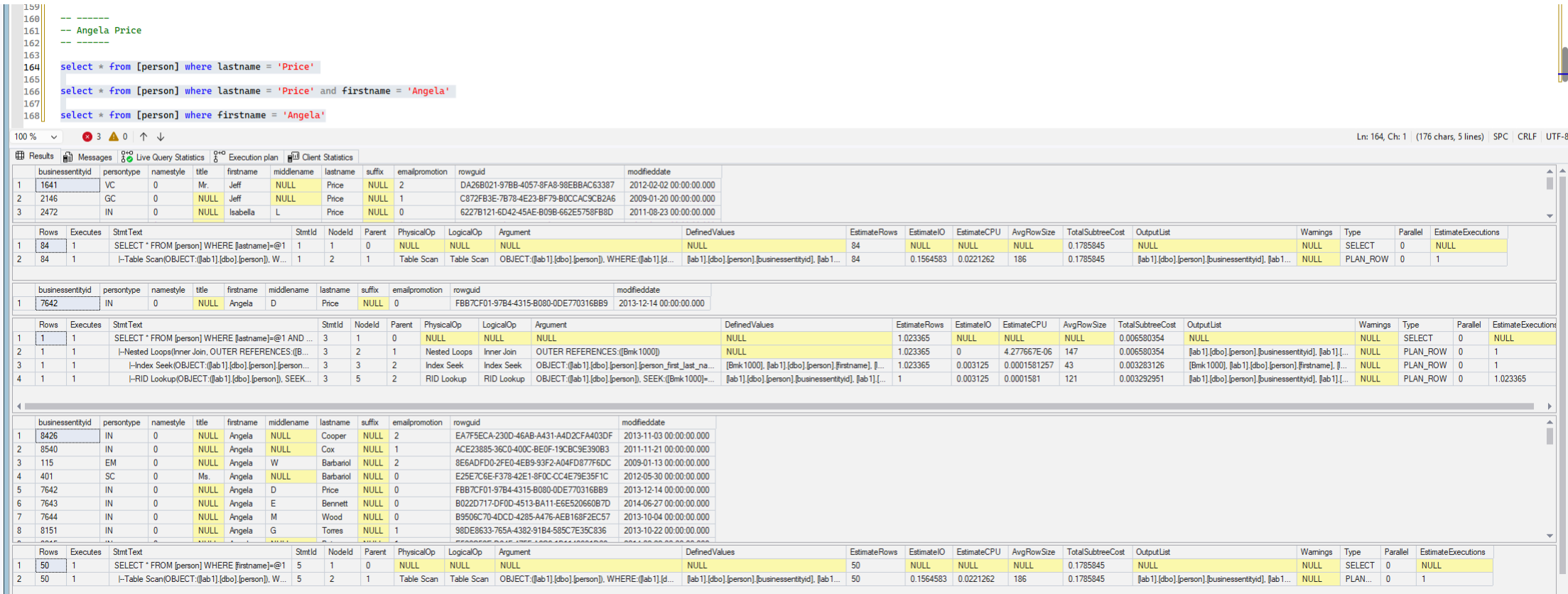


Figure 52: Plany zapytań o Angela Price - indeks na (lastname, firstame)

Komentarz:

W przypadku zapytań 1. oraz 3. (odpowiednio tylko z warunkiem where lastname="..." oraz where firstname="..."), ze względu na ilość osób o odpowiedniu zadanym nazwisku lub imieniu, MSSQL zdecydował że skorzystanie z indeksu (a następnie pobieranie brakujących w indeksie danych ze znalezionych adresów) będzie mniej wydajne niż przeszukiwanie całej tabeli (Table Scan). W przypadku warunku FirstName = ‘Angela’ LastName = ‘Price’ indeks jest wykorzystywany.

Punktacja:

zadanie	pkt
1	2
2	2
3	2
4	2
5	2
razem	10